

NORMALIZACIÓN DE BASES DE DATOS

PROCESO ACUMULATIVO PARA REDUCIR REDUNDANCIAS Y PREVENIR ANOMALÍAS LÓGICAS

La **normalización de bases de datos** es un proceso analítico y acumulativo que consiste en organizar el diseño lógico de un modelo relacional de datos. Sus objetivos principales son **reducir la redundancia de datos** al mínimo y **simplificar las dependencias lógicas** entre las columnas (Pacheco, 2018). El proceso de normalización se aplica de manera incremental y acumulativa, lo que significa que para que una tabla alcance un nivel determinado, debe cumplir estrictamente con las reglas de los niveles anteriores (Pacheco, 2018; Coronel y Rob, 2003).

¿Por qué es fundamental normalizar? Un diseño defectuoso o desnormalizado produce redundancia descontrolada. Esto acarrea **anomalías lógicas graves** durante las operaciones diarias: *anomalías de inserción* (imposibilidad de registrar datos parciales), *anomalías de actualización* (inconsistencia de datos al modificar registros duplicados) y *anomalías de borrado* (pérdida no deseada de información histórica al eliminar un registro relacionado) (Pacheco, 2018; Coronel et al., 2011).

LAS TRES PRIMERAS FORMAS NORMALES (1FN, 2FN, 3FN)

Forma Normal	Requisitos Clave (Reglas Formales)	Dependencias Eliminadas	Estado del Diseño
Primera Forma Normal (1FN)	1. Todos los atributos de las columnas deben ser atómicos (indivisibles). 2. No deben existir grupos o conjuntos repetidos. 3. Cada intersección fila/columna almacena un valor simple. 4. Clave primaria definida.	Evita campos multi-valorados y estructuras no tabulares complejas.	Formato tabular básico estructurado.
Segunda Forma Normal (2FN)	1. Cumplir estrictamente con la 1FN. 2. No deben existir dependencias parciales: todos los atributos no clave deben depender por completo de la clave primaria completa, no de una sección.	Dependencias parciales (donde un atributo depende de solo una parte de una clave primaria compuesta).	Integridad en claves compuestas.
Tercera Forma Normal (3FN)	1. Cumplir estrictamente con la 2FN. 2. No deben existir dependencias transitivas: los atributos que no son clave deben depender de manera exclusiva de la clave primaria, nunca de otra columna no clave.	Dependencias transitivas (atributos no clave que se determinan funcionalmente entre sí).	Estándar ideal de negocios relacionales.

Detalle de las Reglas de Normalización

Primera Forma Normal (1FN): Atomicidad de Valores. La atomicidad exige que cada campo contenga información indivisible. Por ejemplo, almacenar una dirección completa ('Calle 5 de Mayo #12, Col. Centro, CP 06000') en una sola columna plantea graves problemas para realizar búsquedas o clasificaciones. La 1FN obliga a descomponer este dato complejo en atributos elementales como calle, número, colonia y código postal. Asimismo, prohíbe terminantemente la existencia de nidos o celdas repetidas en un mismo registro (Pacheco, 2018; Coronel et al., 2011).

Segunda Forma Normal (2FN): Dependencia Funcional Completa. La 2FN aborda las tablas que poseen claves primarias compuestas (formadas por dos o más columnas). La regla establece que un atributo no clave no puede depender funcionalmente de solo una porción de la clave compuesta. Si una tabla posee una clave primaria simple

(un solo atributo), automáticamente cumple con la 2FN una vez alcanzada la 1FN (Pacheco, 2018; Coronel et al., 2011).

CASO DE ESTUDIO PRÁCTICO: TRANSFORMACIÓN A 3FN

A continuación se muestra un escenario práctico de inscripciones de estudiantes, cursos y profesores.

1. Tabla Inicial Desnormalizada (Con campos divisibles y redundancias)

ID_Estud	Estudiante	Curso_Y_Profesor
10	Ana García	BBDD-D. Pérez, Web-M. Gómez (Campos divisibles, grupos nidados, violando 1FN)
20	Carlos López	BBDD-D. Pérez (Redundancia de nombre de profesor dependiente del curso)

Proceso de descomposición en 3FN:

- Para cumplir con la **1FN**, los atributos se vuelven atómicos y se eliminan grupos repetidos, separando el curso y el profesor.
- Para cumplir con la **2FN**, se elimina la dependencia parcial separando la información de los Estudiantes y los Cursos en entidades distintas.
- Para cumplir con la **3FN**, se elimina la dependencia transitiva. Como el *Profesor* depende del *Curso* y el *Curso* de la clave de la inscripción, el profesor dependía de forma indirecta de la clave. Se crea una tabla *Profesores* independiente para que cada entidad no clave dependa únicamente de su identificador primario (Pacheco, 2018).

2. Estructura de Base de Datos en 3FN (Diseño Final)

Tabla	Clave Primaria (PK)	Atributos y Claves Foráneas (FK)	Ejemplo de Fila
Estudiantes	ID_Estud (Simple)	Estudiante	10 Ana García
Profesores	ID_Prof (Simple)	Profesor	P1 D. Pérez
Cursos	ID_Curso (Simple)	Nombre, ID_Prof (FK a Profesores)	C1 BBDD P1
Inscripciones	ID_Estud, ID_Curso (Compuesta)	ID_Estud (FK a Estudiantes), ID_Curso (FK a Cursos)	10 C1

CÓDIGO SQL: CREACIÓN DE ESQUEMA EN 3FN

```
-- Creación de tablas e integridad referencial
CREATE TABLE Estudiantes (
  ID_Estud INT PRIMARY KEY,
  Estudiante VARCHAR(100) NOT NULL
);

CREATE TABLE Profesores (
  ID_Prof VARCHAR(10) PRIMARY KEY,
  Profesor VARCHAR(100) NOT NULL
);

CREATE TABLE Cursos (
  ID_Curso VARCHAR(10) PRIMARY KEY,
  Nombre VARCHAR(100) NOT NULL,
  ID_Prof VARCHAR(10),
  FOREIGN KEY (ID_Prof) REFERENCES Profesores(ID_Prof)
);

CREATE TABLE Inscripciones (
  ID_Estud INT,
  ID_Curso VARCHAR(10),
  PRIMARY KEY (ID_Estud, ID_Curso),
  FOREIGN KEY (ID_Estud) REFERENCES Estudiantes(ID_Estud),
  FOREIGN KEY (ID_Curso) REFERENCES Cursos(ID_Curso)
);
```

¿CUÁNDO ES RECOMENDABLE LA DESNORMALIZACIÓN?

Aunque la 3FN es considerada la forma ideal de diseño empresarial, las reglas rígidas de normalización **no consideran el rendimiento de procesamiento** de las consultas (Pacheco, 2018). Conforme escalamos hacia formas normales superiores, se crean más tablas. Esto incrementa considerablemente la cantidad de uniones (JOINS) necesarias para recuperar información, lo cual impacta negativamente la velocidad debido a operaciones físicas adicionales de entrada/salida (I/O) en disco y ciclos de CPU (Pacheco, 2018; Coronel y Rob, 2011).

La **desnormalización** es la duplicación intencionada de columnas en varias tablas. Esto sacrifica la integridad de los datos de manera controlada y aumenta la redundancia, pero a cambio **optimiza notablemente la velocidad de lectura** en sistemas con alta demanda de consultas o arquitecturas de Data Warehouse (Pacheco, 2018).

Conclusión: Diseñar bases de datos en 3FN es el punto de partida técnico ideal para proteger la consistencia y evitar anomalías lógicas de manipulación de datos. Sin embargo, el analista debe balancear la pureza teórica del diseño con los requisitos de rendimiento reales de la infraestructura, aplicando desnormalización estratégica cuando la velocidad de consulta sea un factor decisivo.

Fuentes Consultadas:

1. Pacheco, M. (2018). *Normalización de Bases de Datos. Unidad de Apoyo para el Aprendizaje*. CUAED/FCA-UNAM.
2. Coronel, C., Morris, S. y Rob, P. (2011). *Bases de datos. Diseño, implementación y administración*. Cengage Learning.